



CSS152L - Systems Analysis and Design

TECHNICAL DOCUMENTATION

Bayport Veterinary Clinic Management System (VCMS)

Architecture, implementation, deployment, and maintenance guide

Bayport Veterinary Clinic
322 Quirino Avenue, Brgy. Don Galo, Paranaque City
0968 633 2940

Document Version 1.0

June 08, 2026

Document Control

Document Title	Bayport VCMS - Technical Documentation
Version	1.0
Date	June 08, 2026
Prepared For	Bayport Veterinary Clinic staff and stakeholders
System URL (Cloud)	https://bayport-vet-clinic-com.netlify.app
API URL	https://bayport-api.onrender.com/api

Revision history: Version 1.0 - Initial release covering desktop and cloud deployment, role-based workflows, and operational procedures.

Table of Contents

1. Introduction
2. System Overview
3. Architecture
4. Technology Stack
5. Database Design
6. Backend API
7. Frontend Application
8. Security
9. Email and Notifications
10. Deployment - Cloud
11. Deployment - Desktop
12. Environment Variables
13. Maintenance and Monitoring
14. Known Limitations
15. References

1. Introduction

This document describes the technical design and implementation of the Bayport Veterinary Clinic Management System (VCMS). It is intended for developers, system administrators, and IT staff responsible for deployment, integration, and maintenance.

The system follows a three-tier architecture: static web client, REST API server, and relational database. The same codebase supports local desktop packaging (Electron) and cloud hosting.

2. System Overview

Core functional modules:

- Authentication and role-based access control (JWT, optional MFA)
- Pet and owner records with medical history
- Appointment scheduling and workflow (Pending, Approved, Done)
- Clinical consultations: diagnosis, procedures, prescriptions, PDF generation
- Billing, invoicing, and point-of-sale with inventory integration
- Email reminders (templates, scheduled dispatch)
- Administrative reports, audit logs, recycle bin, user management

3. Architecture

Client (HTML/CSS/JavaScript) communicates with Spring Boot REST API over HTTPS. The API persists data via JPA/Hibernate to PostgreSQL (cloud) or H2/MySQL (desktop). File uploads (pet photos) use local disk storage or configurable cloud storage provider.

Logical flow:

1. User browser loads static assets from Netlify (or Electron shell locally).
2. JavaScript resolves API base URL (deploy-env.js or localStorage).
3. Authenticated requests include JWT Bearer token.
4. Spring Boot controllers delegate to service layer and repositories.
5. Flyway migrations apply schema changes on startup.

4. Technology Stack

Layer	Technology	Version / Notes
Frontend	HTML5, Tailwind CSS, vanilla JS	Static pages, Chart.js for reports
Desktop shell	Electron + Node.js	Bundles JRE and backend JAR
Backend	Java, Spring Boot 3	REST, JPA, Flyway, JWT
Database (cloud)	PostgreSQL via Supabase	Session pooler, IPv4
Database (desktop)	H2 embedded file DB	Default; optional MySQL
Hosting (UI)	Netlify	netlify.toml build pipeline
Hosting (API)	Render	Docker, freecloud profile

Email (cloud)	Resend HTTP API	SMTP blocked on Render free tier
Email (desktop)	Gmail SMTP	App password in mail.env

5. Database Design

Primary entities (PostgreSQL tables managed by JPA/Flyway):

Table	Purpose
users, roles	Authentication and RBAC
owners, pets	Client and patient records
pet_medical_records	Historical clinical notes
appointments	Scheduling and status workflow
procedures, prescriptions	Clinical visit data
billing_records, sales, sale_lines	Financial transactions
inventory_items	Stock and catalog
notifications	In-app and reminder queue
audit_log, operation_logs	Compliance and auditing
mfa_code	One-time passcodes

Supabase project reference: `gmyoopqvqpybloqadix`. Use Session pooler host (`aws-1-ap-northeast-2.pooler.supabase.com`) on Render - not the direct IPv6-only host.

6. Backend API

Base path: `/api`. Representative endpoints:

Endpoint	Method	Description
<code>/api/health</code>	GET	Service status, mail configuration
<code>/api/auth/login</code>	POST	Authenticate; may return <code>MFA_REQUIRED</code>
<code>/api/auth/mfa/verify</code>	POST	Verify OTP
<code>/api/pets</code>	GET/POST/PUT	Pet CRUD
<code>/api/appointments</code>	GET/POST/PUT	Appointment management
<code>/api/prescriptions</code>	GET/POST	Prescription records
<code>/api/sales</code>	POST	POS checkout
<code>/api/notifications</code>	GET/POST	Reminders and alerts
<code>/api/admin/users</code>	GET/POST	User administration

Authorization: `@PreAuthorize` and service-layer checks enforce `ADMIN`, `VET`, and `FRONT_OFFICE` roles. CORS is configured via `SPRING_WEB_CORS_ALLOWED_ORIGINS` for cloud deployments.

7. Frontend Application

Key client files:

- assets/api.js - API base resolution, HTTP client, cloud timeout (90s)
- assets/app.js - Role config, sidebar, guard(), login helpers
- assets/deploy-env.js - Generated at Netlify build from BAYPORT_API_BASE
- index.html - Login, MFA, role selection
- Per-module HTML pages with shared header (app-header.js)

Role keys: admin, vet, front_office (UI label: Front Office). Legacy receptionist/pharmacist map to front_office.

8. Security

- JWT tokens stored in localStorage; sent as Authorization Bearer header
- BCrypt password hashing
- Optional MFA (4-digit OTP via email) for non-default accounts
- Device lock: trusted device list in Settings (admin approval)
- Terms of Service acceptance on login
- Audit and data-access logging for sensitive operations
- HTTPS enforced in cloud (Netlify, Render)

9. Email and Notifications

Reminder and MFA emails use EmailService with HTML templates (message-first layout, hosted logo URL). Cloud: Resend API (RESEND_API_KEY, RESEND_FROM). Desktop: Gmail SMTP. If email is not configured, MFA codes appear in admin notifications bell.

10. Deployment - Cloud

1. Supabase: create project, note Session pooler JDBC URL and credentials.
2. Render: deploy bayport-api from GitHub (render.yaml), set freecloud profile and env vars.
3. Netlify: connect repo, set BAYPORT_API_BASE to Render /api URL, trigger deploy.
4. Render CORS: SPRING_WEB_CORS_ALLOWED_ORIGINS must match Netlify URL exactly.
5. Verify: GET /api/health returns status UP; login from Netlify site succeeds.

Production URLs:

- Frontend: <https://bayport-vet-clinic-com.netlify.app>
- API: <https://bayport-api.onrender.com/api>
- Database dashboard: supabase.com/dashboard/project/gmyoopqvqvpybloqadix

11. Deployment - Desktop

1. `cd bayport-vet-clinic/desktop-shell && npm install && npm run desktop`
2. Backend starts on localhost:8080 with H2 database in ./data
3. Optional: configure MySQL in application.properties for Workbench integration
4. Build installers: `npm run verify:win` or `verify:mac`

12. Environment Variables

Variable	Service	Purpose
BAYPORT_API_BASE	Netlify	Render API URL for frontend build
SPRING_DATASOURCE_URL	Render	Supabase JDBC connection string
SPRING_DATASOURCE_USERNAME	Render	postgres.gmyoopqvqvpybloqadix
SPRING_DATASOURCE_PASSWORD	Render	Supabase DB password
SPRING_WEB_CORS_ALLOWED_ORI	Render	Netlify site origin
JWT_SECRET	Render	Token signing key
RESEND_API_KEY	Render	Transactional email (cloud)
RESEND_FROM	Render	Sender address for Resend
BAYPORT_EMAIL_LOGO_URL	Render	Hosted logo for email HTML

13. Maintenance and Monitoring

- Health endpoint: /api/health - monitor mailConfigured and status UP
- Render logs: database connection errors, email failures
- Supabase Table Editor: verify data after app changes (manual refresh)
- Flyway migrations: db/migration/V*.sql applied on startup
- Rotate default passwords and JWT_SECRET before production
- Render free tier: cold start 30-60s after idle - document for users

14. Known Limitations

- Render free tier blocks outbound SMTP; Resend required for email
- Render free tier sleeps after inactivity; first request may be slow
- Cloud file uploads may be ephemeral without attached volume or S3
- Inbox sender avatar requires Gmail profile photo or BIMl + custom domain
- Supabase Realtime disabled by default; not used by application

15. References

- Repository: https://github.com/1una229/bayport-vet-clinic_1
- DEPLOY_ONLINE.md - Netlify + Render + Supabase guide
- RUN_INSTRUCTIONS.md - Desktop setup
- docs/EMAIL_SENDER_AVATAR.md - Email branding and BIMl
- Resend documentation: <https://resend.com/docs>
- Supabase documentation: <https://supabase.com/docs>